
ODIN

Panduan Lengkap

MCP Agent AI untuk Server Linux

Claude Code adalah otak. ODIN adalah tangan.

*Perintah bahasa manusia di laptop, dieksekusi di server Linux yang hidup —
dengan kartu risiko sebelum setiap perubahan.*

Versi 2.3.0

September 2026

Syamsuddin Ideris · syamsuddin.ideris@gmail.com

Lisensi MIT · github.com/Syamsuddin/ODIN

Daftar Isi

1	Mengenal ODIN	4
1.1	Apa itu ODIN	4
1.2	Bentuk konkretnya	4
1.3	Tiga sifat yang menentukan desainnya	5
1.4	Angka-angka pokok	5
1.5	Untuk siapa buku ini	5
2	Manfaat dan Kapan Memakai ODIN	6
2.1	Sepuluh masalah nyata yang diselesaikan	6
2.2	Manfaat menurut peran	6
2.2.1	Bagi operator tunggal (satu orang mengurus semua)	6
2.2.2	Bagi tim kecil	6
2.2.3	Bagi pemilik aplikasi yang bukan sysadmin	7
2.3	Kapan ODIN bukan jawabannya	7
3	Arsitektur dan Cara Kerja	8
3.1	Peta besar	8
3.2	Komponen satu per satu	8
3.2.1	Di server	8
3.2.2	Di laptop	9
3.3	Siklus hidup satu sesi	9
3.4	Multi-project: berpindah = pindah folder	10
4	Fitur Lengkap	11
4.1	Dua puluh tool MCP	11
4.1.1	Eksekusi dan inspeksi	11
4.1.2	Deploy dan pengujian	11
4.1.3	Kecerdasan alur kerja	11
4.1.4	Memory dan cortex	12
4.1.5	Dua resource MCP	12
4.2	Output intelligence — 23 pola error	13
4.3	Rollback tracking	13
4.4	Runbook engine	13
4.5	Pre-flight check dan deteksi drift	14
4.6	Server profiler dan mode operasi	14
4.7	Memory persisten dan Cortex	15
4.8	Orchestrator — sistem saraf otonom	15
4.9	Pembelajaran berkelanjutan	16
4.10	Audit log	16
4.11	Ketahanan eksekusi	16
5	Model Keamanan	18
5.1	Lima lapis pertahanan	18
5.2	Membaca kartu risiko	18
5.3	Lima tier risiko	19
5.4	Mengapa wildcard pada sudoers berbahaya	19

5.5	Kunci SSH yang tidak memberi shell	20
5.6	Perlindungan tambahan	20
5.7	Yang ODIN bukan	21
6	Instalasi dan Konfigurasi	22
6.1	Prasyarat	22
6.2	Pemasangan: dua perintah	22
6.3	Yang dikerjakan installer	22
6.4	Tata letak di laptop	23
6.5	Tata letak di server	23
6.6	odin setup — satu wizard, empat tahap	24
6.7	Dua mode konfigurasi MCP	24
6.8	Memulai pekerjaan	24
6.9	Memutakhirkan dari versi 2.2 ke bawah	25
6.10	Mencopot pemasangan	25
7	Referensi Perintah CLI	26
7.1	Penyiapan	26
7.2	Server	26
7.3	Project	26
7.4	Integrasi Claude Code	27
7.5	Pemeliharaan	27
7.6	Slash command di dalam Claude Code	28
8	Simulasi Penggunaan	29
8.1	Skenario 1 — Pemeriksaan kesehatan sederhana	29
8.2	Skenario 2 — Restart Nginx setelah salah konfigurasi	29
8.3	Skenario 3 — Deploy Laravel dengan deteksi drift	31
8.4	Skenario 4 — Backup database lewat runbook	32
8.5	Skenario 5 — Darurat: disk penuh	32
8.6	Skenario 6 — Berpindah antar project	33
8.7	Skenario 7 — Menyimpan instruksi durable	34
8.8	Skenario 8 — Rollback setelah deploy gagal	35
8.9	Skenario 9 — Pemantauan proaktif	36
8.10	Pelajaran umum dari sembilan skenario	36
9	Pemecahan Masalah	37
9.1	Diagnostik pertama	37
9.2	Tool mcp__odin__* tidak muncul di Claude Code	37
9.3	Koneksi MCP timeout saat startup	37
9.4	odin tidak dikenali setelah instalasi	37
9.5	laravel_deploy diblokir	37
9.6	Perintah baca meminta konfirmasi	38
9.7	Sudoers server masih rentan	38
9.8	Memory terasa penuh atau lambat	38
10	Lampiran A — Variabel Lingkungan	39
11	Lampiran B — Glosarium	41
12	Lampiran C — Batasan yang Diketahui	42

Bagaimana membaca buku ini. Bab 1–2 menjelaskan apa dan mengapa — cukup dibaca sekali. Bab 3–5 adalah rujukan teknis: arsitektur, fitur, dan model keamanan; boleh dilompati saat pertama kali dan dibuka lagi ketika dibutuhkan. Bab 6 membawa Anda dari nol sampai ODIN bekerja. Bab 7 adalah kamus perintah. **Bab 8 berisi sembilan simulasi nyata** — bagian yang paling cepat membuat paham cara kerja ODIN sehari-hari. Bab 9 untuk saat ada yang tidak beres.

Peringatan sebelum mulai. ODIN memberi asisten AI kemampuan mengeksekusi perintah pada server produksi Anda. Buku ini menjelaskan lima lapis pengaman yang membatasi kemampuan itu — tetapi lapis paling penting tetap **Anda**, yang membaca kartu risiko sebelum menekan setuju. Jangan pernah menjalankan ODIN dengan pola pikir “biar AI saja yang memutuskan”.

Mengenai ODIN

1.1 Apa itu ODIN

ODIN adalah **jembatan eksekusi** antara Claude Code — asisten AI yang berjalan di laptop Anda — dan satu atau lebih server Linux yang hidup. Perumpamaan yang paling tepat:

Claude Code adalah otak. ODIN adalah tangan.

Otak memahami maksud Anda, menyusun rencana, membaca hasil, dan memutuskan langkah berikutnya. Tangan yang benar-benar menyentuh server: menjalankan perintah, membaca berkas log, me-restart layanan, mengarahkan aplikasi.

Tanpa ODIN, Claude Code hanya bisa memberi Anda perintah untuk disalin-tempel ke terminal. Dengan ODIN, Claude Code mengeksekusi perintah itu sendiri — dengan izin Anda — lalu membaca keluarannya, menganalisis kegagalan, dan melanjutkan sampai pekerjaan selesai.

Secara teknis ODIN adalah **server MCP** (*Model Context Protocol*) yang berjalan di server Linux Anda dan diakses melalui SSH. Tidak ada port baru yang dibuka, tidak ada daemon yang menyala terus-menerus, tidak ada API key yang disimpan di server.

1.2 Bentuk konkretnya

Seorang operator mengetik satu kalimat di terminal:

```
Cek kenapa website error 500, perbaiki kalau bisa
```

Yang terjadi kemudian:

```
Claude Code (otak)
1. server_info          -> "Project: simuru, Ubuntu 24.04, disk 42%, PHP 8.3"
2. http_health_check    -> "HTTP 500"
3. tail_log laravel.log -> "SQLSTATE[HY000] [2002] Connection refused"
4. ODIN menganalisis    -> {error_type: "db_conn", hints: [...]}
5. service_action mysql -> "inactive (dead)"          <- DITEMUKAN
6. service_action restart mysql
                        -> [KARTU RISIKO SEDANG -> operator menyetujui]
                        -> "active (running)"
7. http_health_check    -> "HTTP 200"          <- SELESAI
```

Total waktu di bawah dua menit. Intervensi manusia: satu kali menyetujui restart MySQL.

1.3 Tiga sifat yang menentukan desainnya

Pertama, ODIN tidak berpendapat soal benar-salah, tetapi soal risiko. Setiap perintah diklasifikasikan: membaca (READ) atau mengubah (WRITE). Perintah baca berjalan otomatis. Perintah tulis selalu memunculkan **kartu risiko** yang harus Anda setuju. Kartu itu memberi tahu tier risikonya, apa yang akan terjadi, seberapa luas dampaknya, dan bagaimana membatalkannya.

Kedua, ODIN mengingat. Memory persisten membuat setiap sesi baru tidak dimulai dari nol. ODIN ingat nama layanan di server Anda, instruksi durable (“selalu backup sebelum deploy”), dan — sejak v2.1 — **pelajaran dari kesalahan sebelumnya.**

Ketiga, ODIN ringan sampai ekstrem. Lima berkas Python, satu dependensi di server (mcp[cli]), tanpa framework, tanpa build step, tanpa daemon. Proses ODIN lahir saat sesi Claude Code dimulai dan mati saat sesi berakhir.

1.4 Angka-angka pokok

Aspek	Nilai
Versi	2.3.0
Baris kode	~7.382 (agent 3.799 · CLI 2.381 · guard 895 · launcher 151 · updater 156)
Test otomatis	771 di 17 berkas, berjalan ~18 detik
Tool MCP	20 (+2 resource)
Perintah CLI	19
Lapis keamanan	5
Dependensi server	1 (mcp[cli])
Dependensi laptop	2 (paramiko, pyyaml) — terisolasi di venv
Overhead server saat mengganggu	0 (tidak ada daemon)

1.5 Untuk siapa buku ini

Buku ini ditulis untuk **operator yang bertanggung jawab atas server produksi** — sysadmin, DevOps, atau pengembang yang merangkap keduanya. Anda diasumsikan sudah terbiasa dengan SSH, systemd, dan baris perintah Linux. Anda **tidak** perlu tahu apa pun tentang MCP Python, atau cara kerja model bahasa.

Kalau Anda hanya ingin memasang ODIN dan mulai bekerja, lompat ke **Bab 6** dan **Bab 8**.

Manfaat dan Kapan Memakai ODIN

2.1 Sepuluh masalah nyata yang diselesaikan

Tanpa ODIN	Dengan ODIN
SSH manual, mengetik perintah satu per satu	Instruksi bahasa manusia, eksekusi otomatis
Lupa urutan deploy, ada langkah terlewat	Alur konsisten lewat runbook engine (maks 20 langkah)
Error tidak terdeteksi sampai pengguna komplain	23 pola error dianalisis seketika + saran perbaikan
Error berulang, debugging ulang dari nol	Pelajaran tersimpan otomatis ke memory
Panik saat rollback: “commit sebelumnya apa?”	State dicatat otomatis sebelum operasi destruktif
Setiap sesi mulai dari nol	Memory persisten: cortex global + memory per project
Kelelahan menyetujui: klik setuju tanpa membaca	Kartu risiko 5 tier — baca sekilas, putuskan cepat
Keadaan server tidak diketahui	Inspeksi otomatis: tipe, stack, mode terdeteksi sendiri
Tidak ada jejak operasi	Audit log append-only + salinan ke journald
Tidak sadar sedang bekerja di project mana	Identitas project di setiap kartu risiko

2.2 Manfaat menurut peran

2.2.1 Bagi operator tunggal (satu orang mengurus semua)

Beban kognitif terbesar seorang operator tunggal bukan mengetik perintah, melainkan **mengingat konteks**: server mana, versi PHP berapa, nama service FPM-nya apa, di mana letak log, apa yang berubah minggu lalu. ODIN memindahkan beban itu ke memory persisten yang dibaca otomatis di awal setiap sesi.

2.2.2 Bagi tim kecil

Runbook dan template membuat prosedur menjadi **eksplisit dan dapat diulang**, bukan pengetahuan yang hanya ada di kepala satu orang. Audit log memberi jawaban atas pertanyaan “siapa mengubah apa, kapan” tanpa harus menelusuri riwayat shell.

2.2.3 Bagi pemilik aplikasi yang bukan sysadmin

Kartu risiko menerjemahkan perintah teknis menjadi kalimat yang bisa dinilai: apa yang dilakukan, apa efeknya, apa saran pengamanannya. Anda tidak perlu memahami `systemctl reload php8.3-fpm` untuk tahu bahwa dampaknya “muat ulang config tanpa memutus koneksi”.

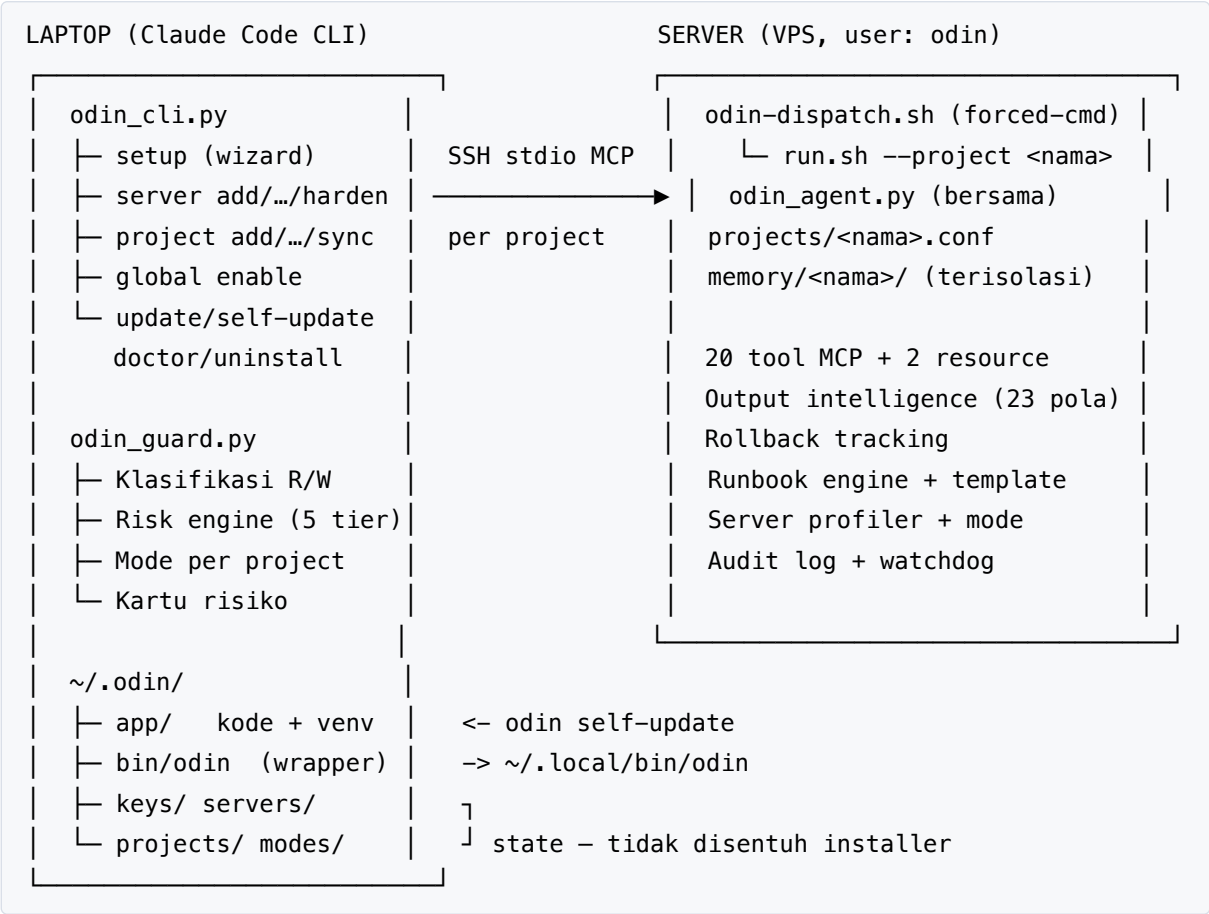
2.3 Kapan ODIN bukan jawabannya

Kejujuran soal batas sama pentingnya dengan daftar manfaat:

- **ODIN bukan sandbox.** Ia tidak mengurung Claude. Batas sesungguhnya adalah hak akses user odin di sistem operasi. Kalau Anda memberi user itu sudo penuh tanpa batas, tidak ada lapis pengaman yang bisa menolong.
- **ODIN bukan pengganti CI/CD.** Untuk pipeline yang berjalan otomatis tanpa manusia, gunakan CI. ODIN dirancang untuk pekerjaan yang **butuh penilaian manusia di tengahnya**.
- **ODIN bukan alat untuk mengelola ratusan server serentak.** Satu sesi = satu project = satu server. Untuk fleet besar, gunakan Ansible atau sejenisnya.
- **ODIN tidak menghilangkan tanggung jawab Anda.** Setiap operasi tulis menunggu persetujuan. Kalau Anda menyetujui tanpa membaca, ODIN akan patuh mengeksekusinya.

Arsitektur dan Cara Kerja

3.1 Peta besar



3.2 Komponen satu per satu

3.2.1 Di server

odin_agent.py (3.799 baris) — inti ODIN. Server MCP dengan transport *stdio* yang berjalan sebagai user `odin`. Ia memuat 20 tool, mesin analisis error, sistem memory, profiler server, dan audit log. Berkas ini **satu untuk semua project**; yang membedakan antar-project hanyalah variabel lingkungan yang diberikan saat peluncuran.

run.sh — peluncur. Menerima `--project <nama>`, membaca `projects/<nama>.conf`, lalu menetapkan `PROJECT_ROOT`, `ALLOWED_LOG_DIRS`, `MEMORY_DIR`, dan `PROJECT_NAME`. Sejak v2.3 ia menolak argumen yang tidak dikenal, dan menolak berjalan bila ada dua project atau lebih tanpa `--project` (dulu diam-diam jatuh ke `/var/www/html` — project yang salah).

odin-dispatch.sh — gerbang SSH (baru di v2.3). Dipasang sebagai *forced-command* di `authorized_keys`, sehingga kunci SSH milik ODIN **hanya** bisa meluncurkan `run.sh [--project <nama>]` dan tidak bisa dipakai untuk mendapatkan shell.

projects/<nama>.conf — konfigurasi per project. Isinya sederhana:

```
PROJECT_NAME=simuru
PROJECT_ROOT=/var/www/simuru
ALLOWED_LOG_DIRS=/var/log,/var/www/simuru
SERVER_ID=a1b2c3d4...    # machine-id, di-seed otomatis (anti mis-route)
```

memory/<nama>/ — memory dan audit log per project. Sengaja diletakkan **di luar** `PROJECT_ROOT` supaya selamat dari `git reset --hard` saat deploy dan tidak terbaca lewat `run_command` maupun `tail_log`.

3.2.2 Di laptop

odin_cli.py (2.381 baris) — perintah odin. Mengelola server, project, konfigurasi Claude Code, pembaruan, dan diagnostik. Memakai `paramiko` untuk SSH saat menyiapkan server.

odin_guard.py (895 baris) — gerbang keputusan. Dipasang sebagai *hook* `PreToolUse` di Claude Code: setiap kali Claude hendak memanggil tool ODIN, guard-lah yang memutuskan apakah langsung dijalankan atau harus memunculkan kartu risiko.

odin_mcp_launch.py (151 baris) — peluncur MCP global. Satu entri konfigurasi melayani semua project: saat Claude Code memulai sesi, launcher mencocokkan direktori kerja dengan daftar project di `~/.odin/projects/` lalu menyambung ke server yang tepat.

update_checker.py (156 baris) — pemeriksa versi di latar belakang.

3.3 Siklus hidup satu sesi

1. Anda menjalankan `claude` di dalam direktori kerja sebuah project.
2. Claude Code membaca konfigurasi MCP, memanggil `odin_mcp_launch.py`.
3. Launcher mencocokkan direktori kerja dengan `~/.odin/projects/*.yaml`, menemukan project dan server-nya, lalu menjalankan `ssh` ke server itu.
4. Di server, kunci SSH memaksa `odin-dispatch.sh` yang hanya mengizinkan `run.sh --project <nama>`.
5. `run.sh` memuat konfigurasi project, memverifikasi identitas mesin (`SERVER_ID`), lalu menjalankan `odin_agent.py`.
6. Agent memuat memory ke dalam instruksi awal sesi, sehingga Claude langsung “ingat” konteks server ini. Bila profil server sudah basi, inspeksi dijalankan di **thread latar** agar tidak memblokir koneksi.
7. Selama sesi, setiap panggilan tool melewati guard di laptop, lalu dieksekusi di server.
8. Saat sesi Claude Code berakhir, koneksi SSH tertutup dan proses agent mati sendiri. *Watchdog* memastikan agent yang kehilangan induknya tidak menjadi proses yatim.

3.4 Multi-project: berpindah = pindah folder

Konsep intinya satu kalimat:

1 project = 1 direktori kerja lokal + 1 server + memory terisolasi + mode operasi sendiri.

Tidak ada perintah “switch project” di dalam sesi Claude Code, karena satu sesi setara dengan satu koneksi MCP yang terikat ke satu project. Berpindah project = pindah folder, lalu buka sesi Claude Code baru. Nama server MCP-nya selalu `odin` (prefiks tool selalu `mcp__odin__*`); yang berubah di belakang layar adalah target SSH dan parameter `--project`.

Isolasi yang berlaku per project: memory, audit log, mode operasi, dan `PROJECT_ROOT`.

Fitur Lengkap

4.1 Dua puluh tool MCP

4.1.1 Eksekusi dan inspeksi

Tool	Fungsi	Persetujuan
run_command	Menjalankan perintah shell apa pun	READ otomatis, WRITE kartu risiko
tail_log	Membaca N baris terakhir berkas log	Otomatis
service_action	Mengelola systemd: status, restart, reload, start, stop	Status otomatis, sisanya kartu risiko
server_info	Ringkasan server: OS, disk, memory, PHP, uptime, nama project	Otomatis
inspect_server	Inspeksi penuh: deteksi tipe, pemindaian stack, derivasi mode	Otomatis

4.1.2 Deploy dan pengujian

Tool	Fungsi	Persetujuan
laravel_deploy	Deploy Laravel satu tombol: git reset, composer, migrate, cache, reload FPM	Kartu risiko TINGGI
run_tests	Menjalankan PHPUnit/Pest	Otomatis
http_health_check	Verifikasi status HTTP + cuplikan body (2.000 byte)	Otomatis

4.1.3 Kecerdasan alur kerja

Tool	Fungsi	Persetujuan
runbook	Menjalankan alur multi-langkah (maks 20) dengan analisis error & rollback per langkah	Kartu risiko tier tertinggi
runbook_templates	Daftar/ambil template bawaan dan kustom	Otomatis
rollback_plan	Saran pembatalan untuk operasi destruktif terakhir	Otomatis
session_history	Riwayat operasi pada sesi berjalan	Otomatis
audit_tail	Membaca audit log dengan penyaring	Otomatis

4.1.4 Memory dan cortex

Tool	Fungsi	Persetujuan
memory_write	Menyimpan fakta/instruksi lintas sesi	Kartu risiko RENDAH
memory_recall	Pencarian semantik TF-IDF + filter namespace/tag	Otomatis
memory_forget	Menghapus entri (tombstone)	Kartu risiko RENDAH
memory_digest	Ringkasan cortex + memory project + event 24 jam	Otomatis
memory_health	Diagnostik: entri basi, duplikat, jumlah per namespace, ukuran	Otomatis
cortex_log	Mencatat peristiwa lintas project	Kartu risiko RENDAH
cortex_events	Membaca jurnal peristiwa lintas project	Otomatis

4.1.5 Dua resource MCP

Resource	Fungsi
memory://{ns}	Membaca memory per namespace tanpa panggilan tool
health://live	Pemeriksaan kesehatan ringan (disk, memory, load, service) untuk polling

4.2 Output intelligence – 23 pola error

Setiap kali sebuah perintah gagal, ODIN memindai keluarannya terhadap 23 pola yang diurutkan dari paling spesifik ke paling umum, lalu melampirkan analisis:

```
{"_analysis": {
  "error_type": "db_conn",
  "hints": ["Cek service MySQL, pastikan berjalan. Periksa .env DB_HOST dan DB_PORT."],
  "suggested_commands": [{"cmd": "systemctl status mysql", "risk": "AMAN"}]}
```

Kategori	Pola yang dideteksi
Database	SQLSTATE (autentikasi, koneksi, constraint, umum), deadlock, max connections
PHP/Laravel	Fatal error, kehabisan memory, timeout, class not found, composer lock
Sistem	Disk penuh, OOM killer, permission denied, command not found
Alat	Nginx config error, sertifikat SSL kedaluwarsa, kegagalan build npm

Delapan belas dari 23 pola menyertakan `suggested_commands` — perintah lanjutan yang sudah dilabeli tingkat risikonya.

Sejak v2.3, `tail_log` menjalankan analisis ini pada **isi berkas log**, bukan hanya ketika exit code bukan nol. Alasannya sederhana: `tail` yang berhasil selalu keluar dengan kode 0, sehingga jalur “baca log lalu deteksi SQLSTATE” dulu tidak pernah terpicu.

4.3 Rollback tracking

Sebelum perintah destruktif (`git reset`, `artisan migrate`, `systemctl restart`, ...):

1. **Tangkap keadaan** — commit hash, status migrasi, status service.
2. **Jalankan perintah.**
3. **Sarankan pembatalan** berdasarkan keadaan yang tadi ditangkap.

```
{"_rollback_hint": ["git reset --hard abc1234",
  "php artisan migrate:rollback --step=1 --force"]}
```

Perintah rollback adalah **saran**, bukan aksi otomatis — operator yang meninjau dan menjalankannya. Tool `rollback_plan` menampilkan ringkasan dari riwayat sesi.

4.4 Runbook engine

Claude menyusun daftar langkah sesuai konteks; ODIN mengeksekusinya berurutan:

- Maksimal 20 langkah per runbook.
- Analisis error dan pelacakan rollback pada setiap langkah.
- Berhenti pada kegagalan pertama, kecuali langkah itu ditandai `continue_on_fail`.
- Laporan terstruktur: `executed / total / skipped / failed`.
- Guard menilai risikonya dari langkah WRITE bertier tertinggi; bila semua langkah bersifat baca, runbook berjalan otomatis.

Empat template bawaan: `ssl-renew`, `db-backup`, `log-cleanup`, `health-check`. Template kustom dapat disimpan ke memory dan akan menimpa template bawaan bernama sama.

4.5 Pre-flight check dan deteksi drift

Sebelum `laravel_deploy` dijalankan, ODIN memeriksa:

Pemeriksaan	Kondisi pemblokir
Pemakaian disk	$\geq 95\%$ → deploy dibatalkan
Berkas git kotor	Ada perubahan belum di-commit → peringatan
Commit saat ini	Dicatat sebagai rujukan rollback
Versi PHP	Diperiksa kompatibilitasnya
Drift	Membandingkan keadaan sekarang dengan sidik jari deploy terakhir

Sidik jari deploy berisi git HEAD, MD5 `composer.lock`, jumlah migrasi, dan jumlah baris `.env`. Bila ada yang berubah di luar jalur deploy — misalnya seseorang menyunting `.env` langsung di server — ODIN melaporkannya sebelum deploy berikutnya berjalan.

4.6 Server profiler dan mode operasi

Pipeline inspeksi terdiri dari lima tahap: inspeksi dasar (OS, kernel, uptime, disk, memory, firewall, fail2ban, SSH, cron), deteksi tipe (web-app, database, container, general), pemindaian stack sesuai tipe, inspeksi aplikasi (`.env`, vendor, framework, git), dan derivasi mode.

Inspeksi tidak pernah memblokir koneksi. Bila profil di memory berumur kurang dari satu jam, profil itu dipakai langsung. Bila tidak, inspeksi dijadwalkan di thread latar dan sesi langsung hidup dengan mode deploy. Ini perbaikan penting di v2.3: sebelumnya inspeksi berjalan saat impor modul dengan total timeout $30+15+30+15$ detik — jauh melewati batas koneksi MCP Claude Code yang 30 detik.

Tiga mode operasi:

Mode	Kapan	Efek di server	Efek di guard
setup	Komponen penting belum ada	Semua diizinkan	Tier normal
deploy	Default	Semua diizinkan	Tier normal

Mode	Kapan	Efek di server	Efek di guard
production	Hanya dari sinyal eksplisit	Blokir laravel_deploy dan pemasangan paket	Tier naik 1 level

Mode production hanya lahir dari sinyal eksplisit: APP_ENV=production|prod|live di .env aplikasi, variabel ODIN_ENV=production, atau override manual di memory. Uptime dan pemakaian disk **bukan** sinyal lingkungan — aturan lama (“uptime lebih dari 7 hari dan disk di bawah 80%”) membuat VPS staging yang hidup delapan hari otomatis dianggap produksi, kehilangan laravel_deploy serta apt/npm/pip install, dan keputusan itu di-cache satu jam.

4.7 Memory persisten dan Cortex

Arsitekturnya dua lapis:

- **Cortex** (global, lintas project) — memory/_cortex/ dengan namespace profile (identitas operator) dan cross (fakta lintas project).
- **Project** (terisolasi) — memory/<project>/ dengan namespace server (fakta infrastruktur) dan instruction (arahan durable dari Anda).

Kemampuannya:

- **Pencarian semantik TF-IDF** pada memory_recall, murni pustaka standar Python.
- **Deteksi basi:** entri berumur lebih dari 30 hari ditandai [STALE?].
- **Deteksi kemiripan:** memory_write memperingatkan bila ada entri serupa (kemiripan Jaccard $\geq 0,45$) — tidak memblokir, hanya memberi tahu.
- **Anggaran digest:** maksimal 3.000 karakter, diprioritaskan profile > instruction > server.
- **Jurnal peristiwa:** deploy dan restart service dicatat lintas project, diringkas 24 jam.
- **Penjaga rahasia:** nilai yang mirip kata sandi, token, kunci privat, JWT, atau AWS key ditolak sebelum masuk berkas.

Penyimpanannya berupa JSONL *append-only* dengan pelipatan *last-write-wins*, tombstone untuk penghapusan, dan TTL. Sejak v2.3, pemadatan dipicu oleh **rasio record mati** (lebih dari 50%) atau **ukuran berkas** (lebih dari 8 MB) — bukan lagi jumlah entri hidup, karena pemicu lama tidak pernah tercapai lewat pembaruan (id-nya tetap sama, hanya versi lamanya menumpuk).

4.8 Orchestrator – sistem saraf otonom

Berjalan otomatis pada enam tool utama, memperkaya hasil dengan empat lapis informasi:

1. **_memory_context** — mengingat kembali memory yang relevan, termasuk pelajaran bertag error-lesson, tanpa Claude harus memanggil memory_recall.
2. **_suggested_next** — deploy sukses menyarankan health check; deploy gagal menyarankan rollback_plan dan tail_log.
3. **_attention** — menandai error berulang dan peristiwa penting dari project lain.
4. **_learned** — hasil pembelajaran otomatis pada panggilan ini.

4.9 Pembelajaran berkelanjutan

Loop	Cara kerja
Error → pelajaran	Error yang terjadi 3 kali atau lebih disimpan sebagai <code>server:error-lesson-{tipe}</code> berisi penyebab, solusi, dan perintah pemicunya
Pelacakan lintas sesi	Frekuensi error bertahan antar-sesi; sejak v2.3 hitungannya meluruh 50% setiap sesi agar status “berulang” tidak menjadi permanen, dan error generik dikeluarkan dari pembelajaran
Sukses → pola	Deploy dan runbook yang berhasil disimpan polanya, dengan masa berlaku 90 hari

Siklus lengkapnya: pada sesi pertama sebuah error muncul tiga kali dan ODIN menyimpan pelajaran. Pada sesi berikutnya error yang sama muncul, orchestrator memanggil kembali pelajaran itu secara otomatis, dan Claude langsung tahu langkah perbaikannya.

4.10 Audit log

Setiap eksekusi tool dicatat ke `audit.jsonl`: waktu, nama tool, ringkasan, keberhasilan, exit code, durasi, mode, dan nama project. Berkasnya *append-only* dan tidak pernah dipadatkan.

Sejak v2.3 setiap catatan **juga dikirim ke journald**. Alasannya jujur: `audit.jsonl` dimiliki oleh user `odin` dan bisa dikosongkan lewat `run_command` yang sama — artinya pihak yang diaudit bisa menghapus jejaknya sendiri. Salinan di `journald` tidak bisa disentuh user `odin`, sehingga forensik pasca-insiden punya sumber kedua.

4.11 Ketahanan eksekusi

Sekumpulan perbaikan v2.3 yang jarang terlihat tetapi menentukan saat keadaan buruk:

Masalah	Perbaikan
Timeout hanya membunuh pembungkus <code>bash</code> ; <code>composer install</code> terus berjalan sebagai proses yatim	Proses dijalankan di grup sendiri dan seluruh grup dimatikan saat timeout
Keluaran non-UTF-8 memicu error dan perintah yang sebenarnya sukses dilaporkan gagal	Dekode dengan penggantian karakter
Keluaran raksasa ditampung utuh di memori <code>cat berkas → sed -i → cat berkas</code> mengembalikan isi lama selama 60 detik	Dialirkan ke buffer kepala/ekor terbatas Cache baca dibatalkan oleh setiap perintah tulis

Masalah	Perbaikan
tail berkas-hilang <code>\ grep x \ \ true</code> keluar dengan kode 0 → sukses palsu	Status tail dipisahkan dari status grep
Baris JSONL terpotong akibat crash menelan record berikutnya	Penambahan memeriksa byte terakhir lebih dulu
Singleton mengirim SIGKILL 1,5 detik setelah SIGTERM ke PID dari berkas yang selamat dari reboot	PID diverifikasi sebagai proses ODIN; SIGTERM ditunggu sampai ~15 detik

Model Keamanan

5.1 Lima lapis pertahanan

```

Lapis 1: Klasifikasi READ/WRITE          (laptop – odin_guard.py)
      13 pembungkus di-unwrap lebih dulu (env, nice, timeout, xargs, ...)
      23 sub-classifier: git, docker, mysql, npm, curl, ufw, nginx, ...
      READ -> jalan otomatis      WRITE -> lanjut ke lapis 2
      |
Lapis 2: Risk engine + kartu risiko        (laptop)
      5 tier: AMAN -> RENDAH -> SEDANG -> TINGGI -> KRITIS
      26 aturan shell + penilai khusus perintah database
      Identitas project tampil sebagai baris PERTAMA setiap kartu
      Operator membaca, lalu menyetujui atau menolak
      |
Lapis 3: Hard-block katastrofik            (server)
      Flag dinormalkan lebih dulu: rm -fr ≡ rm -f -r ≡ rm --recursive --force
      rm -rf /, mkfs, dd of=/dev, fork bomb, poweroff, DROP DATABASE,
      find / -delete – ditolak kecuali allow_dangerous=True
      |
Lapis 4: Kunci SSH forced-command          (server – odin-dispatch.sh)
      authorized_keys: restrict,command="/home/odin/odin-dispatch.sh"
      Kunci ODIN HANYA bisa meluncurkan run.sh [--project <nama>]
      Tanpa shell interaktif, tanpa TTY
      |
Lapis 5: Hak akses sistem operasi          (server)
      User odin dengan sudoers terbatas, tanpa wildcard pada argumen path
      Divalidasi visudo sebelum dipasang
      >>> INI BATAS KEAMANAN YANG SESUNGGUHNYA <<<

```

Prinsipnya: **baca otomatis, tulis wajib konfirmasi, katastrofik ditahan dua kali.** Guard di laptop sengaja dibuat lebih ketat daripada server.

5.2 Membaca kartu risiko

```

Prj    : simuru → vps-app
RISIKO: TINGGI
Cmd    : git reset --hard origin/main

```

```

Dir   : /var/www/simuru
Aksi  : Buang semua perubahan lokal
Efek  : Perubahan belum commit hilang permanen
Saran : Stash dulu jika ada kerja penting
Undo  : git reflog → git reset --hard <commit-sebelumnya>

```

Cara membacanya, baris demi baris:

- **Prj** — project dan server tujuan. **Selalu periksa baris ini lebih dulu.** Inilah pengaman terhadap kesalahan paling mahal: menjalankan perintah yang benar di server yang salah.
- **RISIKO** — tier. AMAN dan RENDAH umumnya reversibel; SEDANG menyebabkan gangguan singkat; TINGGI menghilangkan data atau mengubah skema; KRITIS berpotensi merusak sistem.
- **Cmd / Dir** — perintah persis dan direktori kerjanya.
- **Aksi / Efek** — apa yang dilakukan dan seberapa luas dampaknya.
- **Saran** — langkah pengaman sebelum menyetujui.
- **Undo** — cara membatalkan, bila ada.

Bila project tidak terdaftar di ~/.odin/projects/, kartu menambahkan peringatan. Sifatnya informatif saja dan tidak memblokir eksekusi.

5.3 Lima tier risiko

Tier	Contoh	Sifat
AMAN	SELECT, ls, systemctl status	Hanya membaca
RENDAH	mkdir, cp, git add, systemctl reload	Reversibel, tanpa downtime
SEDANG	systemctl restart, chmod, apt install, UPDATE ... WHERE	Gangguan singkat atau perubahan terbatas
TINGGI	rm -rf, git reset --hard, DROP TABLE, TRUNCATE	Kehilangan data, sulit dibatalkan
KRITIS	rm -rf /, mkfs, DROP DATABASE, fork bomb	Merusak sistem; server menolak tanpa allow_dangerous

Dalam mode production, semua tier dinaikkan satu tingkat dan kartu menambahkan peringatan MODE PRODUCTION.

5.4 Mengapa wildcard pada sudoers berbahaya

Ini pelajaran keamanan terpenting dari v2.3, dan berlaku umum di luar ODIN.

`sudo` mencocokkan argumen dengan `fnmatch(3)` **tanpa** opsi `FNMPATHNAME`. Akibatnya tanda `*` ikut cocok dengan garis miring dan titik-dua-titik. Aturan yang terlihat aman ini:

```
odin ALL=(root) NOPASSWD: /usr/bin/tail -n * /var/log/*
```

sesungguhnya mengizinkan:

```
sudo -n /usr/bin/tail -n 1 /var/log/../../../../etc/shadow
```

— yaitu membaca berkas apa pun di sistem sebagai root, dan itu dapat dijangkau lewat `run_command` biasa. Empat aturan berikut karena itu dihapus di v2.3:

Aturan lama	Akibatnya
<code>tail -n * /var/log/*</code>	Membaca berkas apa pun sebagai root
<code>certbot renew *</code>	<code>--deploy-hook='...'</code> dieksekusi sebagai root
<code>/usr/local/bin/*-deploy</code>	Menjalankan skrip apa pun bernama demikian, yang bisa ditulis user lain
<code>journalctl *</code>	Tanpa <code>--no-pager</code> , pager berjalan sebagai root dan <code>!/bin/sh</code> di dalamnya memberi shell root

Sejak v2.3, berkas `sudoers` **divalidasi dengan `visudo -cf` sebelum dipasang** dan dibatalkan bila tidak sah — karena `sudoers` yang rusak membuat seluruh `sudo` di server tidak berfungsi.

5.5 Kunci SSH yang tidak memberi shell

Sebelum v2.3, kunci ODIN ditambahkan ke `authorized_keys` apa adanya dan user `odin` punya shell `/bin/bash`. Siapa pun yang memegang kunci privat itu mendapat shell interaktif lengkap dengan TTY — dan dari sana, `sudo journalctl` bisa dipakai membuka pager sebagai root, lalu `!/bin/sh` di dalam pager memberi shell root.

Sekarang barisnya berbentuk:

```
restrict,command="/home/odin/odin-dispatch.sh" ssh-ed25519 AAAA... odin-vps-app
```

Dispatcher hanya menerima `run.sh` dengan opsi `--project <nama>`; nama project dibatasi huruf, angka, titik, garis bawah, dan tanda hubung. Segala metakarakter shell ditolak.

5.6 Perlindungan tambahan

- **Pembungkus perintah di-unwrap** — 13 pembungkus (`env`, `nice`, `ionice`, `nohup`, `timeout`, `stdbuf`, `setsid`, `command`, `builtin`, `exec`, `xargs`, `time`, `unbuffer`) dilewati beserta flag dan penetapan variabelnya, sehingga `env rm -rf /var/www/app` dinilai sebagai `rm`, bukan sebagai `env`. Sebelum v2.3 perintah itu lolos sebagai `READ` dan **dieksekusi tanpa konfirmasi**.
- **Substitusi perintah dan proses** (`$(...)`, backtick, `<(...)`, `>(...)`) selalu memicu konfirmasi, karena isinya menyembunyikan perintah sesungguhnya.

- **Normalisasi flag** — `rm -fr`, `rm -f -r`, dan `rm --recursive --force` dikenali sama dengan `rm -rf` sebelum pencocokan pola katastrofik. Fungsi ini ada di **kedua sisi** (guard dan server) dan harus tetap identik.
- **Pengikatan identitas server** — `SERVER_ID` (machine-id) terikat pada konfigurasi project. Bila koneksi nyasar ke server lain, ODIN menolak berjalan dengan pesan jelas alih-alih diam-diam bekerja di tempat yang salah.
- **Memory di luar webroot** — tidak dapat diakses lewat web, tidak terhapus oleh `git reset --hard`, dan tidak terbaca oleh `run_command` maupun `tail_log`.

5.7 Yang ODIN bukan

ODIN bukan sandbox. Guard dan hard-block adalah jaring pengaman, bukan kurungan. Seorang penyerang yang sudah menguasai laptop Anda bisa memanggil tool ODIN secara langsung. Yang benar-benar membatasi kerusakan adalah hak akses user `odin` di sistem operasi server — karena itu berikan `sudoers` seminimal mungkin, dan jangan menambahkan aturan `berwildcard path`.

Instalasi dan Konfigurasi

6.1 Prasyarat

Di **laptop**: git, Python 3.10 atau lebih baru, klien SSH, dan Claude Code CLI.

Di **server**: Linux dengan systemd, akses SSH sebagai root atau user dengan sudo (diperlukan sekali saja saat penyiapan), serta Python 3 dengan modul venv.

6.2 Pemasangan: dua perintah

macOS dan Linux

```
curl -fsSL https://raw.githubusercontent.com/Syamsuddin/ODIN/main/install.sh | bash
odin setup
```

Windows (PowerShell)

```
irm https://raw.githubusercontent.com/Syamsuddin/ODIN/main/install.ps1 | iex
odin setup
```

Skrip Windows mencerminkan versi Linux baris demi baris, tetapi **belum diuji pada mesin Windows sungguhan**. Setelah memasang, jalankan `odin doctor` untuk memverifikasi.

6.3 Yang dikerjakan installer

Tahap	Rincian
Prasyarat	Memeriksa git, Python ≥ 3.10 , ssh; memperingatkan bila Claude Code belum ada
Versi	Mengambil tag rilis terbaru dari GitHub, bukan main terkini. Dapat dikunci: <code>ODIN_VERSION=v2.3.0</code>
Kode	Meng-clone ke <code>~/.odin/app/</code> sebagai checkout git yang bisa diganti versinya
Dependensi	venv terisolasi di <code>~/.odin/app/.venv/</code> — kebal terhadap PEP 668 (Debian 12, Ubuntu 23.04+, Homebrew)

Tahap	Rincian
Perintah	~/odin/bin/odin dengan symlink ~/local/bin/odin; PATH ditambahkan ke rc shell setelah minta izin
Claude Code	Menyalin slash command /odin:* ke ~/claude/commands/odin/
Verifikasi	Menjalankan odin --version dan mengimpor paramiko/pyyaml sungguhan

Installer bersifat **idempoten** — menjalankannya lagi sama dengan memperbarui — dan **tidak pernah memakai sudo**.

Opsi yang tersedia: --yes (tanpa interaksi), --no-setup, --version <ref>, --home <dir>. Lewat pipa gunakan curl ... | bash -s -- --yes.

6.4 Tata letak di laptop

Kode dan **state dipisah**, sehingga installer maupun pembaruan tidak punya alasan menyentuh data milik Anda:

```
~/odin/
├─ app/                kode ODIN (checkout git)  <- odin self-update
│   └─ .venv/          dependensi CLI
├─ bin/odin            wrapper CLI
├─ keys/               kunci SSH per server
├─ servers/ projects/  registry                  }
├─ modes/  ssh_config  mode per project, entri SSH } milik Anda –
└─ client/ server/ -> app/ symlink kompatibilitas } tidak disentuh
~/local/bin/odin -> ~/odin/bin/odin
```

Pemisahan ini menutup sebuah kegagalan nyata pada versi lama: ~/odin dulu menampung kode **dan** kunci SSH sekaligus, sehingga installer yang menemukan folder non-git akan memin-dahkan seluruhnya — termasuk kunci privat — dari jalur yang masih ditunjuk ~/.ssh/config, dan LogLevel QUIET menyembunyikan keagalannya.

Pengguna versi 2.2 ke bawah dimigrasikan otomatis: hanya berkas yang dilacak git yang dipin-dahkan; sisa yang tidak dikenali dipindah ke .legacy-<timestamp>, bukan dihapus.

6.5 Tata letak di server

```
/home/odin/
├─ odin_agent.py        agent MCP (mode 600)
├─ run.sh               peluncur per project (755)
└─ odin-dispatch.sh     gerbang forced-command (755)
```



```

├─ .venv/                mcp[cli]
├─ projects/<nama>.conf  konfigurasi per project
├─ memory/<nama>/       memory + audit log per project (mode 700)
│   └─ memory.jsonl
│   └─ audit.jsonl

```

6.6 odin setup – satu wizard, empat tahap

```

[1/4] Server      -> odin server add: membuat user odin, memasang sudoers
                    tervalidasi, venv + mcp[cli], mengunggah agent, memasang
                    kunci forced-command, lalu handshake MCP
                    (dilewati bila server sudah terdaftar)
[2/4] Project     -> odin project add, direktori kerja = folder saat ini
                    (dilewati bila folder ini sudah terdaftar)
[3/4] Claude Code -> MCP odin global + hook guard + daftar izin read-only
[4/4] Verifikasi  -> handshake MCP sungguhan ke project ini

```

Setiap tahap idempoten, jadi wizard aman dijalankan berulang. Untuk project kedua dan seterusnya: masuk ke foldernya, jalankan `odin setup` lagi, dan server yang sudah ada akan dipakai ulang.

6.7 Dua mode konfigurasi MCP

Global (disarankan, dipilih otomatis oleh wizard). Satu entri di `~/.claude.json` melayani semua project; `odin_mcp_launch.py` menentukan project dari direktori kerja saat sesi dimulai. Project baru tidak butuh konfigurasi per-repo.

Per direktori kerja. Entri odin ditulis ke `.claude/settings.json` di dalam folder project.

Keduanya memakai hook guard dengan matcher `mcp__odin__.*`, dan hook milik Anda yang sudah ada akan **digabung, bukan ditimpa** — perbaikan penting di v2.3, karena versi sebelumnya menghapus hook Bash milik pengguna tanpa peringatan.

6.8 Memulai pekerjaan

```

cd ~/PROJECTS/SIMURU
claude

```

Di dalam Claude Code, ketik `/odin:status` untuk melihat identitas project dan keadaan server. Ingat bahwa **Claude Code memuat MCP saat memulai** — sesi yang sedang berjalan tidak akan melihat perubahan konfigurasi; bukalah sesi baru.

6.9 Memutakhirkan dari versi 2.2 ke bawah

Ini bagian yang **tidak boleh dilewati** bila server Anda dipasang dengan ODIN lama. Perbaikan sudoers tidak sampai secara otomatis: `server add` melewati sudoers bila berkasnya sudah ada, dan `odin update` berjalan sebagai user `odin` yang tidak berhak menulis ke `/etc/sudoers.d`.

```
odin self-update           # perbarui ODIN di laptop
odin update <alias>        # perbarui agent, run.sh, dispatcher di server
odin server harden <alias> # WAJIB: sudoers aman + kunci forced-command
odin doctor <alias>        # verifikasi: sudoers dan kunci harus OK
```

`odin server harden` meminta kredensial admin, mencadangkan sudoers lama, memasang aturan baru yang tervalidasi, memasang dispatcher, lalu membatasi kunci — dan **memverifikasinya lewat koneksi SSH baru**, karena `forced-command` hanya berlaku saat autentikasi sehingga sesi yang sedang berjalan tidak membuktikan apa pun. Bila verifikasi gagal, `authorized_keys` dikembalikan seperti semula; tidak ada risiko terkunci dari server.

6.10 Mencopot pemasangan

```
odin uninstall             # hapus kode, wrapper, entri Claude Code; state DIPERTAHANKAN
odin uninstall --purge     # hapus juga kunci SSH dan registry
```

Sebelum `--purge`, cabut dulu kunci dari server: `odin server remove <alias> --purge`. Bila CLI sudah rusak, gunakan `curl -fsSL .../uninstall.sh | bash`.

Referensi Perintah CLI

Sembilan belas perintah, dikelompokkan menurut kegunaannya.

7.1 Penyiapan

Perintah	Fungsi
<code>odin setup</code>	Wizard lengkap: server → project → Claude Code → verifikasi. Idempoten
<code>odin version</code>	Versi, lokasi kode, dan lokasi state

7.2 Server

Perintah	Fungsi
<code>odin server add</code>	Menyiapkan server baru (interaktif: host, port, user admin, kata sandi)
<code>odin server list</code>	Daftar server terdaftar
<code>odin server test <alias></code>	Uji koneksi, kesehatan ODIN, dan handshake MCP
<code>odin server harden <alias></code>	Memasang ulang sudoers aman + kunci forced-command pada server lama
<code>odin server remove <alias> [--purge]</code>	Menghapus server; --purge mencabut kunci dari <code>authorized_keys</code>

7.3 Project

Perintah	Fungsi
<code>odin project add</code>	Menautkan direktori kerja lokal ke <code>server:project</code>
<code>odin project list</code>	Daftar project, dengan penanda project aktif
<code>odin project status [nama]</code>	Validasi konfigurasi lokal + ping SSH ke server

Perintah	Fungsi
<code>odin project switch <nama></code>	Membuka tab terminal baru di direktori kerja project
<code>odin project sync [nama\ --all]</code>	Membangun ulang konfigurasi lokal dari manifest
<code>odin project remove <nama></code>	Menghapus konfigurasi project

Pendaftaran tanpa interaksi, berguna untuk skrip:

```
odin project add --name ekampus --server vps-app \
  --remote-root /var/www/ekampus --workdir ~/PROJECTS/EKAMPUS --yes
```

7.4 Integrasi Claude Code

Perintah	Fungsi
<code>odin global enable [--migrate]</code>	Memasang MCP odin untuk semua project; <code>--migrate</code> membersihkan entri per-folder lama
<code>odin global disable</code>	Mencabut entri MCP global

7.5 Pemeliharaan

Perintah	Fungsi
<code>odin doctor</code>	Diagnostik laptop : Python, dependensi, PATH, MCP, hook, server & project
<code>odin doctor <alias></code>	Diagnostik server + handshake MCP per project + audit postur keamanan
<code>odin self-update [ref]</code>	Memperbarui ODIN di laptop ke tag rilis terbaru, atau ref tertentu
<code>odin update <alias></code>	Memperbarui agent di server : backup → uji kompilasi → ganti atomik → handshake
<code>odin uninstall [--purge]</code>	Mencopot ODIN dari laptop

Bedakan dua perintah pembaruan. `self-update` untuk laptop, `update <alias>` untuk server. Keduanya perlu dijalankan setelah rilis baru.

7.6 Slash command di dalam Claude Code

Perintah	Fungsi
<code>/odin:status</code>	Identitas project, keadaan server, ringkasan memory
<code>/odin:doctor</code>	Diagnostik dari dalam sesi
<code>/odin:setup</code>	Panduan konfigurasi ulang
<code>/odin:check-update</code>	Memeriksa versi terbaru
<code>/odin:help</code> , <code>/odin:about</code>	Bantuan dan informasi

Simulasi Penggunaan

Sembilan skenario berikut memperlihatkan ODIN dalam pekerjaan sehari-hari. Keluaran tool disingkat seperlunya agar mudah dibaca.

8.1 Skenario 1 – Pemeriksaan kesehatan sederhana

```
Operator:
cek kesehatan server simuru, terutama disk dan nginx

Claude:
[Tool: mcp__odin__server_info]                               <- auto-allow (read-only)
{"hostname": "vps-app", "os": "Ubuntu 22.04.3 LTS",
  "uptime": "45 days", "disk_usage": {"/": "62%"},
  "memory": {"percent": 52}, "load_avg": "0.45, 0.52, 0.48"}

[Tool: mcp__odin__run_command] systemctl status nginx <- auto-allow (READ)
active (running) since Mon 2026-05-02; 45 days ago

Server SIMURU sehat:
- Disk 62% (aman), memory 52%, load rendah
- Nginx aktif selama 45 hari
```

Yang perlu diperhatikan: tidak ada satu pun kartu risiko muncul. Semua perintah di sini bersifat baca, dan lapis pertama meloloskannya secara otomatis. Inilah alasan pemeriksaan rutin terasa cepat.

8.2 Skenario 2 – Restart Nginx setelah salah konfigurasi

```
Operator:
saya baru edit config nginx, tolong test dan reload

Claude:
[Tool: mcp__odin__run_command] nginx -t                               <- auto-allow (READ)
nginx: [emerg] unknown directive "server_nam" in /etc/nginx/sites-enabled/simuru:12
nginx: configuration file test failed

[_analysis] error_type: nginx_config
  hints: Config Nginx error. Jalankan: nginx -t untuk detail.
```

JANGAN reload sebelum config valid.

Ada salah ketik di baris 12: "server_nam" seharusnya "server_name".
Saya TIDAK akan reload karena config belum valid. Perbaiki dulu?

Operator:

ya perbaiki

Claude:

[Tool: mcp__odin__run_command]

sed -i 's/server_nam /server_name /' /etc/nginx/sites-enabled/simuru

Prj : simuru → vps-app

RISIKO: SEDANG

Cmd : sed -i 's/server_nam /server_name /' /etc/nginx/sites-enabled/simuru

Aksi : Menyunting berkas di tempat (sed -i)

Efek : Isi berkas berubah permanen tanpa salinan

Saran : Pastikan pola cocok; simpan salinan bila ragu

<- operator menyetujui

[Tool: mcp__odin__run_command] nginx -t

syntax is ok / test is successful

[Tool: mcp__odin__service_action] reload nginx

Prj : simuru → vps-app

RISIKO: RENDAH

Cmd : systemctl reload nginx

Aksi : Muat ulang config tanpa memutus koneksi (tanpa downtime)

Undo : systemctl reload nginx (ulangi)

<- operator menyetujui

Nginx berhasil di-reload. Config valid.

Yang perlu diperhatikan: ODIN menolak melanjutkan ke reload selama nginx -t gagal. Analisis error tidak hanya menyebut masalahnya, tetapi juga memberi larangan yang tepat. Perhatikan juga bahwa sed -i dikenali sebagai operasi tulis — sejak v2.3 bentuk panjang sed --in-place juga ikut dikenali.

8.3 Skenario 3 – Deploy Laravel dengan deteksi drift

Operator:

deploy branch main ke simuru

Claude:

[Tool: mcp__odin__laravel_deploy] app_path=/var/www/simuru branch=main

Pre-flight:

```
disk 62% ..... OK
git status ..... 2 berkas termodifikasi <- PERHATIAN
commit sekarang ..... a1b2c3d
PHP ..... 8.3.6 OK
drift ..... .env berubah di luar deploy (48 -> 51 baris)
```

Prj : simuru → vps-app

RISIKO: TINGGI

App : /var/www/simuru Branch: main

Aksi : Deploy Laravel – git reset --hard → composer install --no-dev →
artisan migrate --force → optimize + cache → reload php8.3-fpm

Efek : Perubahan lokal server HILANG; skema DB bisa berubah

Saran : Pastikan backup DB tersedia; deploy saat trafik rendah

Sebelum menyetujui, dua hal perlu Anda tahu:

1. Ada 2 berkas termodifikasi di server yang akan HILANG oleh git reset --hard
2. .env bertambah 3 baris sejak deploy terakhir – kemungkinan disunting manual; perubahan itu TIDAK akan hilang (.env tidak dilacak git), tetapi patut dicek

Lanjutkan?

Operator:

tunjukkan dulu 2 berkas itu

Claude:

[Tool: mcp__odin__run_command] cd /var/www/simuru && git status --short
M app/Http/Controllers/PaymentController.php
M resources/views/invoice.blade.php

Keduanya berkas aplikasi yang seharusnya datang dari git. Kemungkinan besar seseorang menyunting langsung di server – perubahan ini akan hilang.

Saya sarankan simpan dulu salinannya sebelum deploy.

Yang perlu diperhatikan: deteksi drift adalah fitur yang jarang terasa sampai ia menyelamatkan Anda. ODIN membandingkan keadaan server dengan sidik jari deploy terakhir dan memberi tahu

apa yang berubah di luar jalur resmi. Perhatikan juga bahwa Claude **berhenti dan bertanya** alih-alih menyetujui sendiri.

8.4 Skenario 4 – Backup database lewat runbook

Operator:

backup database simuru sebelum maintenance nanti malam

Claude:

[Tool: mcp__odin__runbook_templates] name=db-backup

Template: 4 langkah (dump, verifikasi ukuran, kompres, bersihkan backup lama)

[Tool: mcp__odin__runbook] name=db-backup

Prj : simuru → vps-app

RISIKO: RENDAH

Aksi : Runbook 'db-backup' – 4 langkah

Langkah: dump → verify → compress → cleanup

Efek : Semua langkah dijalankan berurutan; berhenti pada kegagalan pertama

← operator menyetujui

[1/4] dump OK (2.3s) simuru-20260908-2140.sql (48 MB)

[2/4] verify OK (0.1s) ukuran > 1 MB, header valid

[3/4] compress OK (4.1s) simuru-20260908-2140.sql.gz (7.2 MB)

[4/4] cleanup OK (0.2s) 2 backup > 14 hari dihapus

Backup selesai: /var/backups/simuru/simuru-20260908-2140.sql.gz (7,2 MB)

Runbook sukses disimpan sebagai pola: server:success-runbook-db-backup

Yang perlu diperhatikan: satu kartu risiko untuk empat langkah — tier ditentukan oleh langkah paling berisiko. Kalau semua langkah bersifat baca, runbook berjalan tanpa konfirmasi sama sekali.

8.5 Skenario 5 – Darurat: disk penuh

Operator:

website down, kayaknya disk penuh

Claude:

[Tool: mcp__odin__run_command] df -h

← auto-allow

/dev/vda1 40G 39G 0 100% /

[Tool: mcp__odin__run_command] du -sh /var/log/* | sort -rh | head -5

```
22G /var/log/nginx/access.log
1.2G /var/log/syslog
```

Penyebabnya jelas: access.log nginx 22 GB – logrotate tampaknya tidak berjalan.

Rencana saya, berurutan dari yang paling aman:

1. Kosongkan access.log dengan truncate (aman, nginx tetap menulis ke fd yang sama)
2. Jalankan logrotate secara paksa
3. Periksa kenapa logrotate tidak jalan

```
[Tool: mcp__odin__run_command] truncate -s 0 /var/log/nginx/access.log
```

```
Prj    : simuru → vps-app
```

```
RISIKO: SEDANG
```

```
Cmd    : truncate -s 0 /var/log/nginx/access.log
```

```
Aksi   : Mengosongkan berkas log
```

```
Efek   : Isi log hilang permanen; nginx tetap menulis tanpa perlu restart
```

```
Saran  : Pastikan log tidak sedang dibutuhkan untuk investigasi
```

<- operator menyetujui

```
[Tool: mcp__odin__run_command] df -h
```

```
/dev/vda1 40G 17G 21G 45% /
```

```
[Tool: mcp__odin__http_health_check] https://simuru.example.com
```

```
HTTP 200 (0.42s)
```

Disk turun ke 45%, website kembali normal.

Akar masalahnya: /etc/logrotate.d/nginx tidak punya direktif "su www-data adm", sehingga logrotate gagal senyap sejak update paket. Perbaiki sekarang?

Yang perlu diperhatikan: ODIN memilih truncate alih-alih rm karena menghapus berkas log yang sedang dibuka nginx tidak akan mengembalikan ruang disk sampai layanan di-restart. Perhatikan juga urutan kerjanya: memulihkan layanan dulu, mencari akar masalah kemudian.

8.6 Skenario 6 – Berpindah antar project

```
$ cd ~/PROJECTS/SIMURU && claude
/odin:status
Project   : simuru → vps-app (deploy)
Server    : Ubuntu 22.04, uptime 45d, disk 45%
Ingatan   : 14 entri aktif (3 instruksi)

$ cd ~/PROJECTS/eKAMPUS && claude
/odin:status
```

```
Project   : ekampus → vps-kampus (production)
Server    : Debian 12, uptime 8d, disk 31%
Ingatan   : 6 entri aktif (1 instruksi)
```

Yang perlu diperhatikan: tidak ada perintah “switch” — cukup pindah folder dan buka sesi baru. Memory, audit log, dan mode operasi keduanya terpisah penuh. Project kedua berada di mode production, sehingga tier risikonya naik satu tingkat dan `laravel_deploy` diblokir sampai mode diturunkan secara sadar.

8.7 Skenario 7 – Menyimpan instruksi durable

Operator:

```
ingat ya, untuk project ini selalu backup database sebelum deploy,
simpan di /var/backups/simuru
```

Claude:

```
[Tool: mcp__odin__memory_write] ns=instruction key=backup-before-deploy
```

```
Prj      : simuru → vps-app
```

```
RISIKO: RENDAH
```

```
Aksi     : Simpan memory [instruction] key=backup-before-deploy
```

```
Efek     : Entri tersimpan; muncul di konteks sesi berikutnya
```

```
Isi      : Selalu backup database ke /var/backups/simuru sebelum menjalankan
           laravel_deploy pada project ini.
```

← operator menyetujui

```
Tersimpan. Instruksi ini akan otomatis saya baca di setiap sesi berikutnya
untuk project simuru.
```

Pada sesi berikutnya, tanpa diminta:

Operator:

```
deploy branch main
```

Claude:

```
Sesuai instruksi tersimpan, saya backup database dulu sebelum deploy.
```

```
[Tool: mcp__odin__run_command] mysqldump --single-transaction ...
```

Yang perlu diperhatikan: inilah yang membedakan ODIN dari sekadar eksekutor perintah. Instruksi disuntikkan ke konteks awal setiap sesi, jadi Anda tidak perlu mengulanginya.

8.8 Skenario 8 – Rollback setelah deploy gagal

Operator:

deploy tadi bikin error 500, rollback

Claude:

[Tool: mcp__odin__rollback_plan] last=5 <- auto-allow

Operasi destruktif terakhir:

1. laravel_deploy (3 menit lalu)

pre-state: git HEAD a1b2c3d, migrasi 47, php8.3-fpm active

rollback : git reset --hard a1b2c3d

php artisan migrate:rollback --step=1 --force

systemctl reload php8.3-fpm

Rencana rollback ke commit a1b2c3d. Migrasi terakhir juga akan dibatalkan. Sebelum itu – mau saya lihat dulu error 500-nya? Bisa jadi penyebabnya bukan kode, dan rollback tidak akan menolong.

Operator:

ya cek dulu

Claude:

[Tool: mcp__odin__tail_log] /var/www/simuru/storage/logs/laravel.log grep=ERROR
production.ERROR: SQLSTATE[42S22]: Column not found: 'invoices.paid_at'

[_analysis] error_type: db_constraint

Penyebabnya migrasi yang tidak lengkap: kode baru memakai kolom paid_at, tetapi migrasi penambah kolom itu gagal di tengah jalan. Dua pilihan:

A. Rollback penuh ke a1b2c3d (aman, kembali ke keadaan sebelumnya)

B. Jalankan ulang migrasi yang gagal (lebih cepat, tetapi berisiko bila migrasi itu sudah setengah berjalan)

Saya sarankan A karena migrasi setengah jalan sulit dipastikan keadaannya.

Yang perlu diperhatikan: rollback_plan bekerja karena ODIN menangkap keadaan **sebelum** operasi destruktif dijalankan. Perhatikan juga bahwa Claude menahan diri dari rollback membabi buta dan mendiagnosis lebih dulu.

8.9 Skenario 9 – Pemantauan proaktif

Operator:

```
/loop 5m pantau kesehatan server, kabari kalau ada anomali
```

Claude:

```
[Resource: health://live]                                <- polling tiap 5 menit
status: OK          disk: 45%   memory: 52%   load: 0.4
nginx: active       mysql: active   php-fpm: php8.3-fpm
```

```
... (12 siklus tenang, tidak ada laporan) ...
```

```
[Resource: health://live]
status: ANOMALI  disk: 87%   memory: 54%
issues: DISK 87% >= 85%
```

PERHATIAN: disk naik dari 45% ke 87% dalam satu jam.

```
[Tool: mcp__odin__run_command] du -sh /var/lib/mysql/* | sort -rh | head -3
31G  /var/lib/mysql/simuru/sessions.ibd
```

Tabel sessions membengkak 31 GB. Kemungkinan besar session driver database tanpa pembersihan terjadwal. Perlu saya periksa lebih lanjut?

Yang perlu diperhatikan: resource health://live sengaja dibuat sangat ringan sehingga aman dipanggil berulang. Tidak ada daemon dan tidak ada port tambahan — Claude yang melakukan polling.

8.10 Pelajaran umum dari sembilan skenario

1. **Operasi baca tidak pernah mengganggu Anda.** Diagnosis berjalan lancar; konfirmasi hanya muncul saat ada yang benar-benar berubah.
2. **Kartu risiko selalu menyebut project lebih dulu.** Biasakan membaca baris pertama.
3. **ODIN menahan diri saat prasyarat belum terpenuhi** — tidak me-reload nginx dengan config rusak, tidak rollback sebelum mendiagnosis.
4. **Analisis error mengubah keluaran mentah menjadi langkah berikutnya.**
5. **Memory membuat instruksi cukup diucapkan satu kali.**

Pemecahan Masalah

9.1 Diagnostik pertama

Hampir semua masalah terjawab oleh dua perintah ini:

```
odin doctor           # sisi laptop
odin doctor <alias>   # sisi server + handshake MCP sungguhan
```

9.2 Tool mcp__odin__* tidak muncul di Claude Code

Kemungkinan	Pemeriksaan	Perbaikan
Sesi lama masih berjalan	MCP dimuat saat start	Tutup sesi, buka claude baru
MCP belum terdaftar	odin doctor baris “MCP odin global”	odin global enable
Folder belum terdaftar	odin project list	odin setup di folder itu
Kunci SSH bermasalah	odin server test <alias>	odin server harden <alias>
Agent rusak di server	odin doctor <alias> baris handshake	odin update <alias>

9.3 Koneksi MCP timeout saat startup

Pada v2.3 hal ini seharusnya tidak lagi terjadi, karena inspeksi server dipindahkan ke thread latar. Bila tetap terjadi, biasanya penyebabnya di jaringan atau SSH: uji dengan ssh <alias> echo ok. Bila SSH meminta kata sandi atau passphrase, koneksi MCP akan menggantung — entri SSH yang ditulis ODIN memakai BatchMode yes justru agar kegagalan seperti ini cepat terlihat, bukan menggantung.

9.4 odin tidak dikenali setelah instalasi

PATH belum termuat. Buka terminal baru, atau muat ulang berkas rc shell Anda. Periksa dengan odin doctor pada baris “~/local/bin di PATH”.

9.5 laravel_deploy diblokir

Server berada di mode production. Ini disengaja. Bila memang perlu deploy:

```
memory_write(ns='server', key='mode-override', text='deploy')
```

Bila server itu sebenarnya bukan produksi, periksa APP_ENV di berkas .env aplikasi — sejak v2.3 hanya nilai itulah (atau ODIN_ENV) yang membuat ODIN menganggapnya produksi.

9.6 Perintah baca meminta konfirmasi

Guard sengaja konservatif. Substitusi perintah \$(...), substitusi proses <(...), pengalihan keluaran ke berkas, dan perintah yang tidak dikenal semuanya memicu konfirmasi. Ini bukan kesalahan — pada perintah majemuk, guard menilai bagian **paling berisiko**.

9.7 Sudoers server masih rentan

odin doctor <alias> melaporkannya. Perbaikan hanya bisa dilakukan dengan kredensial admin:

```
odin server harden <alias>
```

9.8 Memory terasa penuh atau lambat

```
memory_health()
```

Menampilkan entri basi, duplikat, jumlah per namespace, dan ukuran berkas. Pemadatan berjalan otomatis, tetapi entri yang tidak lagi benar sebaiknya dihapus dengan memory_forget.

Lampiran A – Variabel Lingkungan

Ditetapkan di `projects/<nama>.conf` pada server, kecuali disebutkan lain.

Variabel	Default	Keterangan
PROJECT_NAME	—	Nama project (ditetapkan <code>run.sh</code>)
PROJECT_ROOT	—	Root aplikasi di server
ALLOWED_LOG_DIRS	<code>/var/log,/var/www,/home,/tmp</code>	Folder yang boleh dibaca <code>tail_log</code>
ODIN_ENV	—	<code>production/prod/live</code> menandakan mode produksi
LOCK_CWD_TO_PROJECT	0	1 memvalidasi argumen <code>cwd</code> (bukan konfinemen)
DEFAULT_TIMEOUT	180	Timeout default per perintah (detik)
MAX_TIMEOUT	900	Batas atas timeout
OUTPUT_LIMIT	20000	Ambang pemotongan keluaran (karakter)
CONTEXT_BUDGET	5000	Ambang peringkasan keluaran menjadi kepala/ekor
CACHE_TTL_READ	60	Masa berlaku cache hasil perintah baca; 0 mematikan
MEMORY_DIR	<code>\$ODIN_HOME/memory/<project></code>	Folder memory
MEMORY_MAX_TEXT	4000	Panjang maksimum teks satu entri
MEMORY_MAX_ENTRIES	2000	Ambang pemadatan berdasarkan jumlah entri hidup
MEMORY_MAX_BYTES	8388608	Ambang pemadatan berdasarkan ukuran berkas
MEMORY_DEAD_RATIO	0.5	Rasio record mati yang memicu pemadatan
STALE_DAYS	30	Umur entri sebelum ditandai basi

Variabel	Default	Keterangan
DIGEST_BUDGET	3000	Batas karakter ringkasan memory saat startup
EVENTS_DIGEST_HOURS	24	Jendela peristiwa lintas project
LOG_ROTATE_BYTES	5242880	Ambang rotasi audit.jsonl dan events.jsonl
AUDIT_ENABLED	1	0 mematikan audit log
AUDIT_SYSLOG	1	0 mematikan salinan audit ke journald
SERVER_ID	—	machine-id server, disemai otomatis (anti mis-route)
ODIN_SKIP_INSPECT	0	1 melewati inspeksi startup (untuk pengujian)

Di sisi laptop: ODIN_HOME (default ~/.odin), ODIN_VERSION (tag yang dipasang), ODIN_INSTALL_DIR (lokasi kode, default ~/.odin/app).

Lampiran B – Glosarium

Agent — proses `odin_agent.py` yang berjalan di server dan menyediakan tool MCP.

Cortex — lapisan memory global yang dibagikan lintas project, berisi profil operator dan fakta lintas project.

Drift — perbedaan antara keadaan server sekarang dan sidik jari deploy terakhir; menandakan ada perubahan di luar jalur deploy resmi.

Forced-command — opsi `authorized_keys` yang memaksa sebuah kunci SSH hanya menjalankan satu program tertentu, apa pun perintah yang diminta klien.

Fold — proses melipat log JSONL append-only menjadi keadaan terkini, dengan aturan tulisan terakhir menang dan tombstone untuk penghapusan.

Guard — hook `PreToolUse` di laptop yang mengklasifikasikan perintah dan memunculkan kartu risiko.

Hard-block — penolakan di sisi server terhadap perintah katastrofik, terlepas dari keputusan guard.

Kartu risiko — ringkasan terstruktur (tier, aksi, efek, saran, undo) yang ditampilkan sebelum operasi tulis.

MCP — *Model Context Protocol*, protokol yang menghubungkan asisten AI dengan alat luar. ODIN memakai transport *stdio* di atas SSH.

Mode operasi — `setup`, `deploy`, atau `production`; menentukan seberapa ketat pembatasan berlaku.

Runbook — rangkaian langkah bernama yang dieksekusi berurutan dengan analisis error dan pelacakan rollback.

Tombstone — penanda penghapusan logis pada log append-only.

Lampiran C – Batasan yang Diketahui

Daftar ini sengaja dicantumkan agar Anda tidak menemukan kejutan di lapangan.

Batasan	Keterangan
Installer Windows belum diuji	<code>install.ps1</code> mencerminkan versi Linux baris demi baris, tetapi belum dijalankan pada mesin Windows sungguhan. Verifikasi dengan <code>odin doctor</code> setelah memasang
Server lama tidak diperbaiki otomatis	Perbaikan <code>sudoers</code> memerlukan <code>odin server harden <alias></code> dengan kredensial admin
Satu sesi = satu project	Tidak ada perpindahan project di tengah sesi Claude Code
Bukan sandbox	Batas sesungguhnya adalah hak akses user <code>odin</code> di sistem operasi
<code>LOCK_CWD_TO_PROJECT</code> bukan konfinemen	Hanya memvalidasi argumen <code>cwd</code> ; <code>cd</code> di dalam string perintah tetap berjalan
Log root-only	Aturan <code>sudoers</code> untuk <code>tail</code> sengaja dihapus. Bila Anda perlu membaca log yang hanya bisa dibaca root, buat wrapper khusus yang memvalidasi path — jangan mengembalikan wildcard

ODIN v2.3.0 — multi-server, multi-project, berbasis direktori kerja. Pembelajaran berkelanjutan. Orchestrator. Cortex. Kunci SSH tanpa shell. Instalasi dua perintah.

Kode sumber, changelog, dan laporan masalah: <https://github.com/Syamsuddin/ODIN>

Lisensi MIT · Dibuat oleh Syamsuddin Ideris